COMP9900 – Computer Science Project

No.116 - Paediatric Cancer Methylation

Classifier Model Generation

Final Report

Team: 9900F16B - DONUT

Full Name	Email Address	Student ID	Role
Xiang Li	z5614297@ad.unsw.edu.au	z5614297	Scrum Master
Yizhen Yan	z5640115@ad.unsw.edu.au	z5640115	Product Owner
Long Liu	z5557989@ad.unsw.edu.au	z5557989	Data Engineer
Sandra Li	z5498528@ad.unsw.edu.au	z5498528	DevOps Engineer
Qiliang Ye	z5306314@ad.unsw.edu.au	z5306314	Machine Learning Engineer
Yize Li	z5430483@ad.unsw.edu.au	z5430483	Machine Learning Engineer

Report Submission Date: 25 November 2025

1 Installation Manual 2

1.1 Databricks workspace preparation 3

1.2 Necessary documents 3

1.3 Operation steps 3

1.3.1 Create job 3

1.3.2 Adjust the parameters in job_manager 5

1.3.3 Run Task 6

1.3.4 Waiting for task completion 7

1.3.5 View the model information of nodes that have completed training 7

2 System Architecture Diagram 9

2.1 Key components on Databricks 9

2.2 Workflows of the system 9

2.2.1 Parallel modelling execution workflow 10

2.2.2 End-to-end single node modelling workflow 11

3 Design Justifications 11

4 User-Driven Evaluation of Solution 13

4.1 Main Features of the System 14

4.1.1 Node-Isolated Training Workflow 14

4.1.2 Differential Methylation Feature Selection 14

4.1.3 Hyperparameter Search and Model Training 14

4.1.4 Parallel Execution Through Databricks Jobs 15

4.1.5 Structured Evaluation and Model Registry 15

4.1.6 Prediction for New Methylation Samples 15

4.2 Evaluation Criteria 15

4.3 Evaluation of Effectiveness 16

4.4 Unmet or Partially Met Objectives 17

5 Limitations and Future Work 17

5.1 Limitations 17

5.2 Future Work 17

Table of Contents

1 Installation Manual

This project does not include local deployment or Docker runtime requirements, and all training, evaluation, and model management are completed internally on the Databricks platform. This installation manual explains how to run a complete cancer methylation classification system in Databricks.

1.1 Databricks workspace preparation

The system runs in a Databricks notebook environment, without the need for local installation or Docker.

Please ensure that: You have access to Databricks Workspace.

1.2 Necessary documents

Path / File	Description
sprint3/	Project root directory
sprint3/data/	Data folder
sprint3/data/freeze0525/	Frozen reproducibility snapshot
sprint3/data/freeze0525/diseaseTree_mapped.joblib	Frozen disease tree
sprint3/data/freeze0525/README.md	Metadata freeze documentation
sprint3/data/freeze0525/...	Additional sample metadata
sprint3/disease_tree.py	Disease tree implementation
sprint3/make_tasks_from_tree.py	Script to generate per-node tasks (tasks.joblib)
sprint3/job_manager.py	Databricks Jobs orchestrator for parallel node training
sprint3/parallel_training.py	Core training logic for a single node
sprint3/node_modelling.ipynb	**Notebook executed by each Databricks Job run**
sprint3/load_model_predict_for_new_samples.py	Prediction script for new methylation profiles
sprint3/env_and_paths.py	Environment variables and path configuration

1.3 Operation steps

1.3.1 Create job

Select from the left-hand menu of Databricks Jobs & Pipelines->Create->job

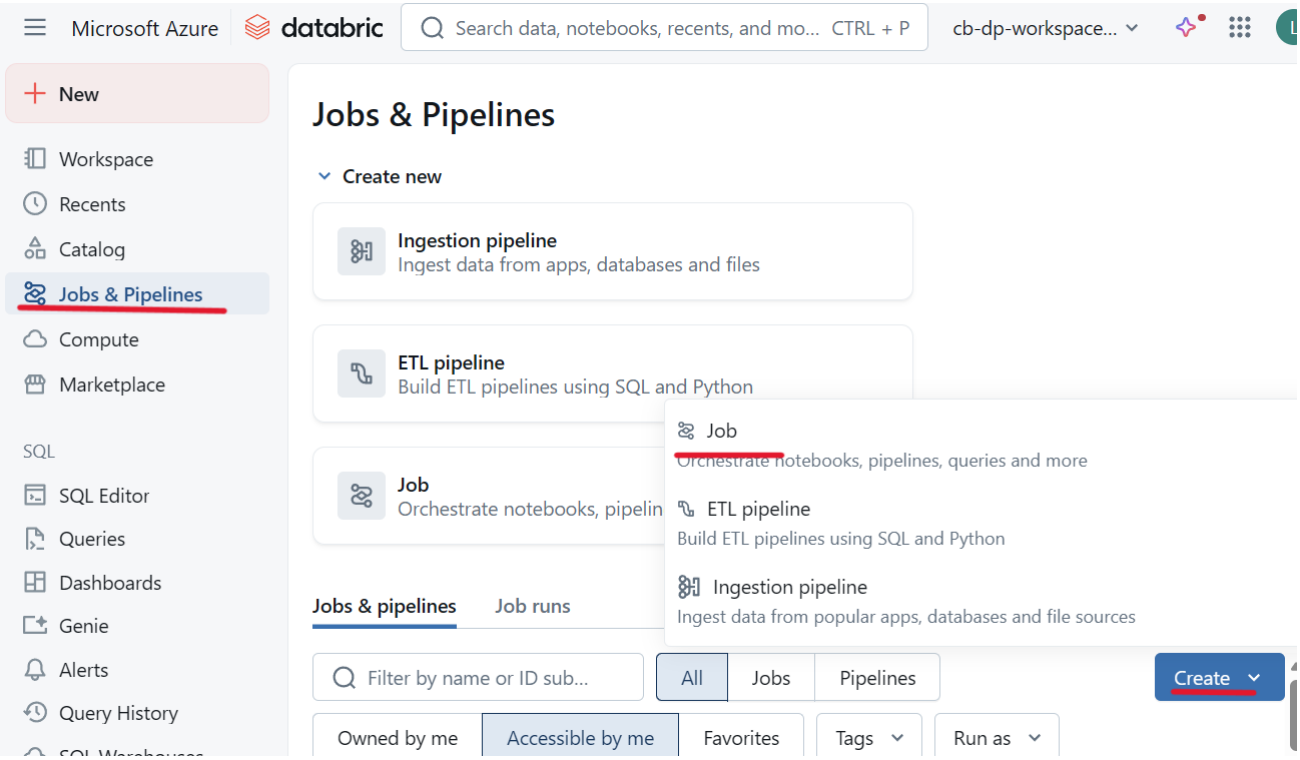

Figure 1 Creating Task

Fill in the path of the notebook to be executed each time the job is run, job ID and other necessary parameters.

In the task, it is necessary to specify the path of the notebook(node_modelling.ipynb) for end-to-end training and the allocated computing resources.

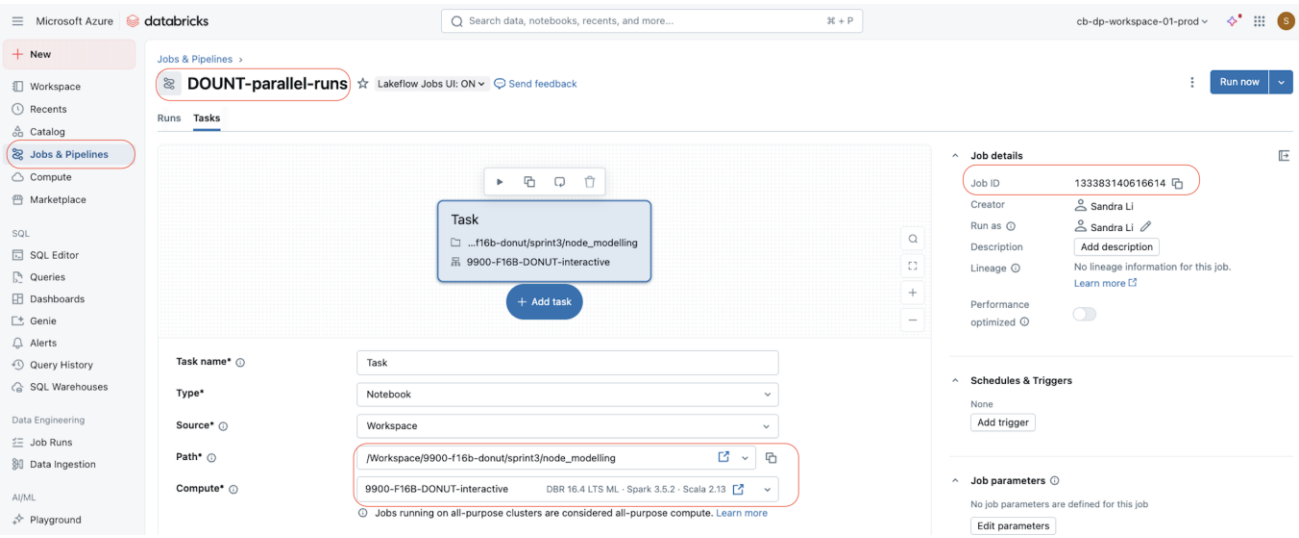

Figure 2 Job & Pipeline task setup

On the parameter interface, set the parameters.

Fill in the selected experiment path in the value corresponding to the key to experiment.

Fill in the top-level parent node name to be trained in the value corresponding to key of node_name. If the node corresponding to node_name does not meet the training criteria, training will not be conducted.

Create key parent_run_id, but leave the corresponding value blank.

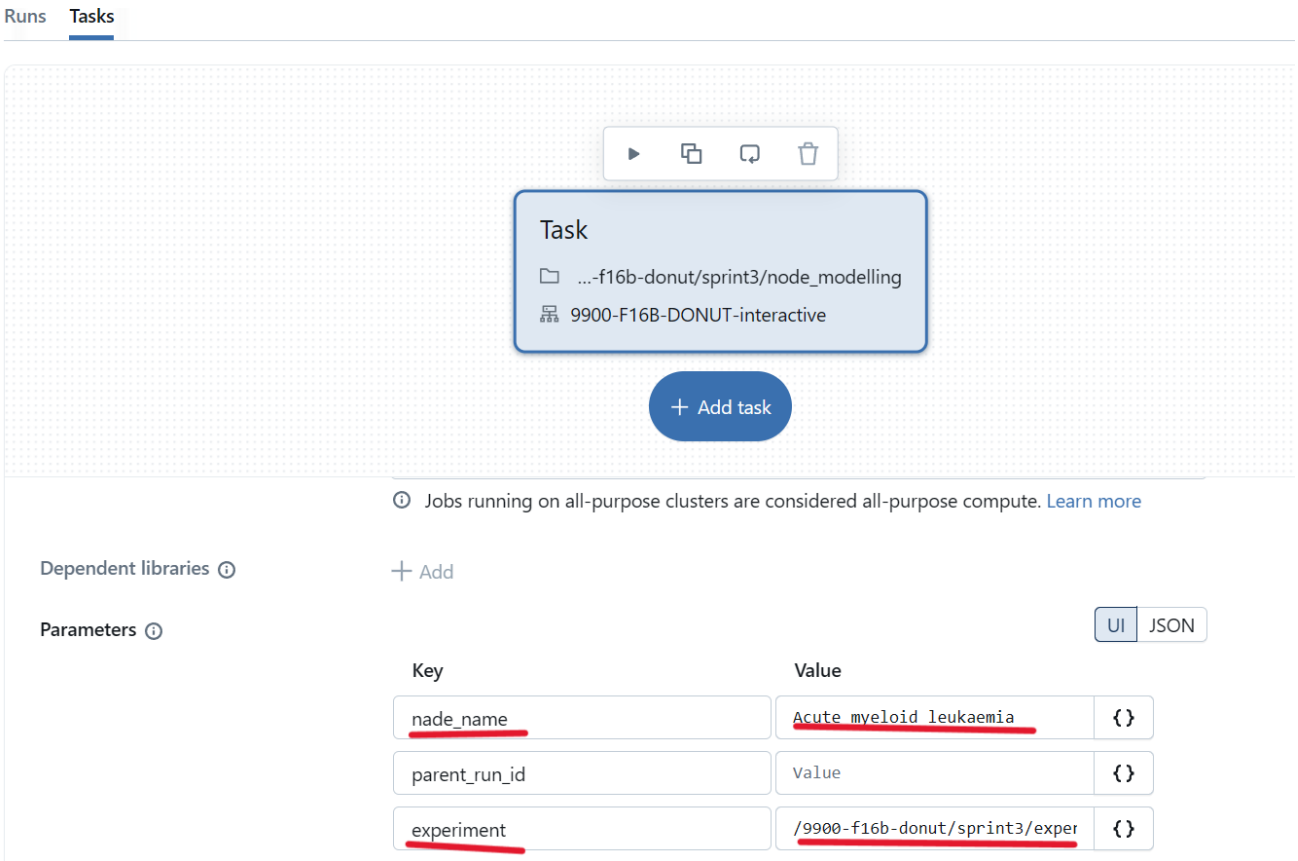

Figure 3 Task Interface

1.3.2 Adjust the parameters in job_manager

Path of job_manager： workspace -> 9900-f16b-dount -> sprint3 -> job_manager

Ensure that the EXPRESS-PATH and RUN_JOB_ID are consistent with the Task created in the previous step.

Adjust the maximum number of parallel nodes run through MAX-PALLEL.

Adjust seconds to wait between polling job run statistics through POLL_INTERVAL.

```
19 hours ago (<1s)                                                    3

# ========= Configuration ===========

# 1. Set node names manually
NODE_LIST = [
    "Acute myeloid leukaemia",
    "Leukaemia"
]

# 2. Set Databricks job parameters and SDK client
EXPERIMENT_PATH = "/9900-f16b-donut/sprint3/experiment"
mlflow.set_experiment(EXPERIMENT_PATH)
RUN_JOB_ID = 133383140616614  # Job ID for the node training
MAX_PARALLEL = 2            # maximum number of parallel node runs
POLL_INTERVAL = 30            # Seconds to wait between polling job run statuses
TERMINAL_STATES = {"TERMINATED", "INTERNAL_ERROR", "SKIPPED"}
```

Figure 4 Job manager configuration setup

1.3.3 Run Task

Select from the left-hand menu of Databricks Jobs & Pipelines.

Select the Task created in the first step. Click the Run now button in the upper right corner.

Click the Runs button in the upper left corner to switch to the running status interface.

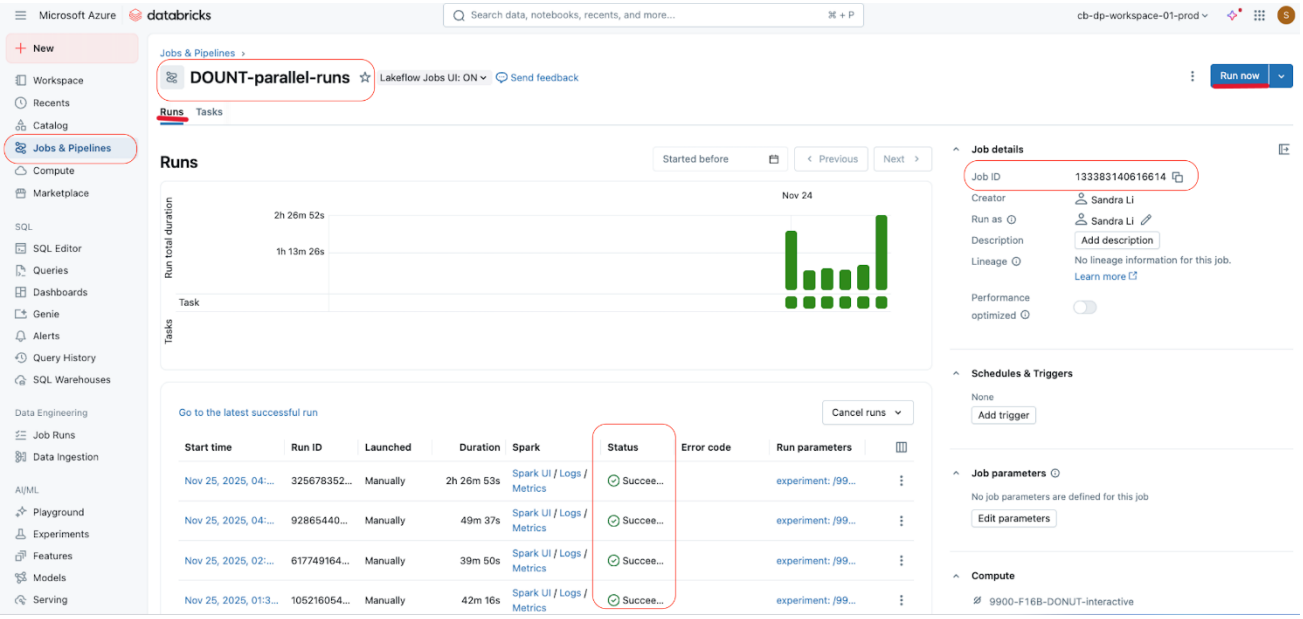

Figure 5 Job running status

1.3.4 Waiting for task completion

1.3.5 View the model information of nodes that have completed training

Select from the left-hand menu of Databricks Models-> Registered Models-> Workspace Model Registry-> Node name to be queried

Click on the node name you want to query.

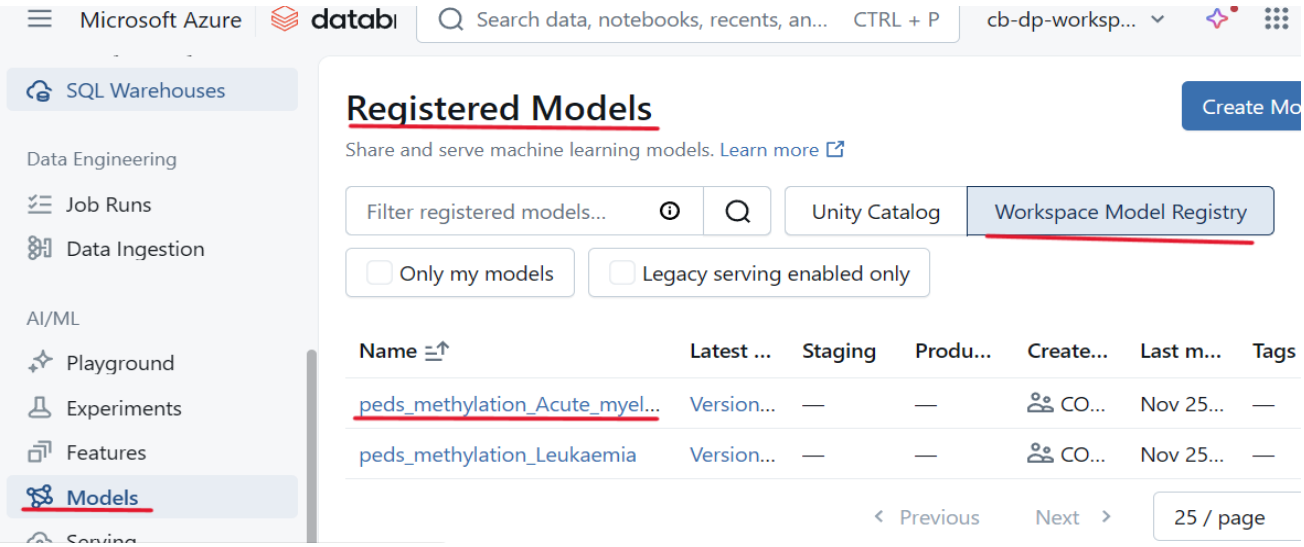

Figure 6 Registered models

Click on the latest version.

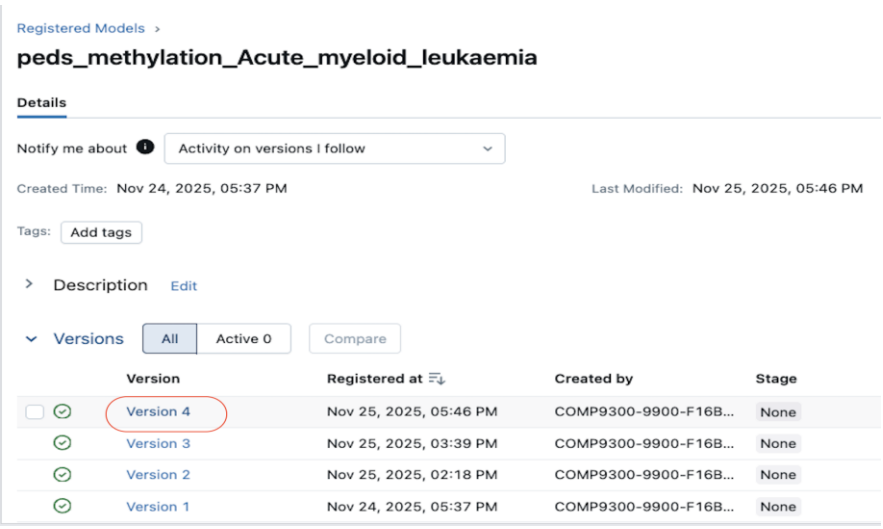

Figure 7 Trained models for a node

Click on the run information corresponding to the version in the upper left corner

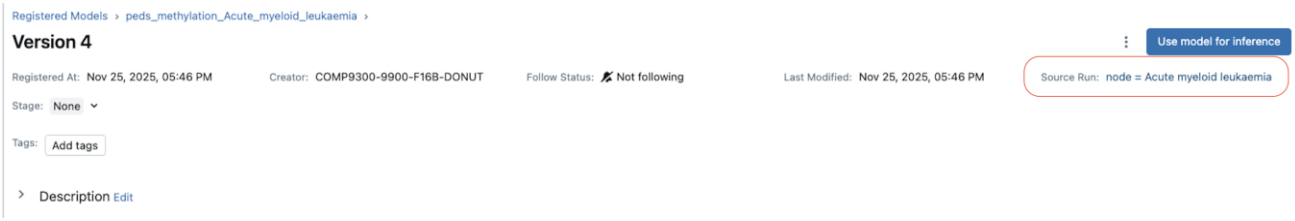

Figure 8 Trained model source run information

Click on Model Metrics to view the model evaluation.

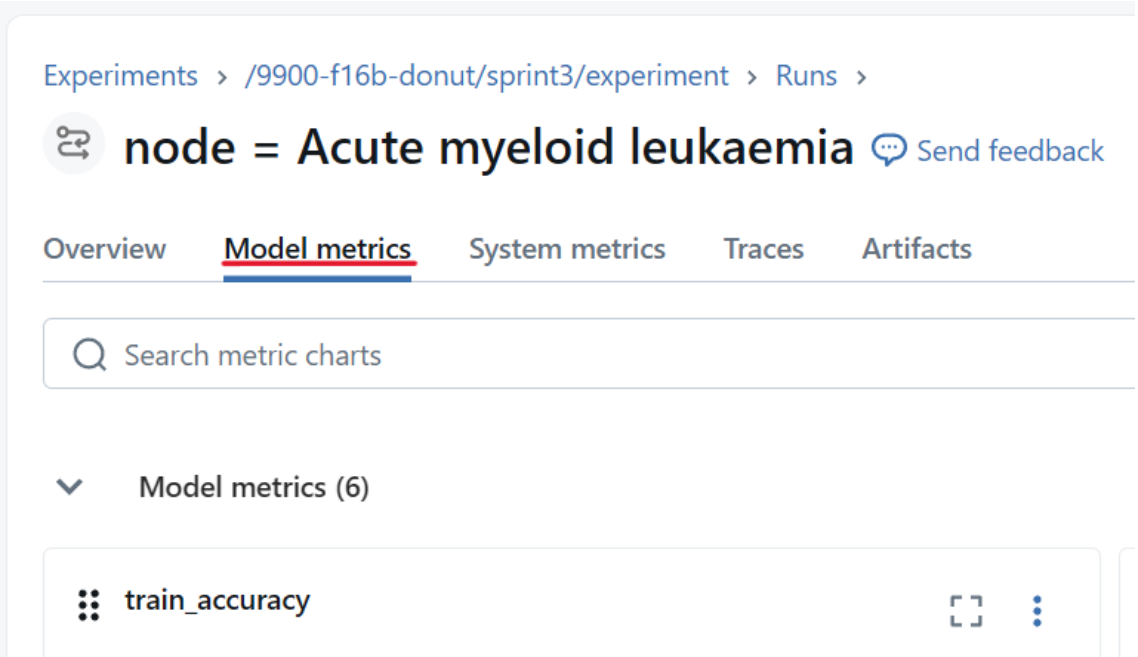

Figure 9 Trained model metrics

Click on Artifacts->eval->.json file to view hyperparameters settings.

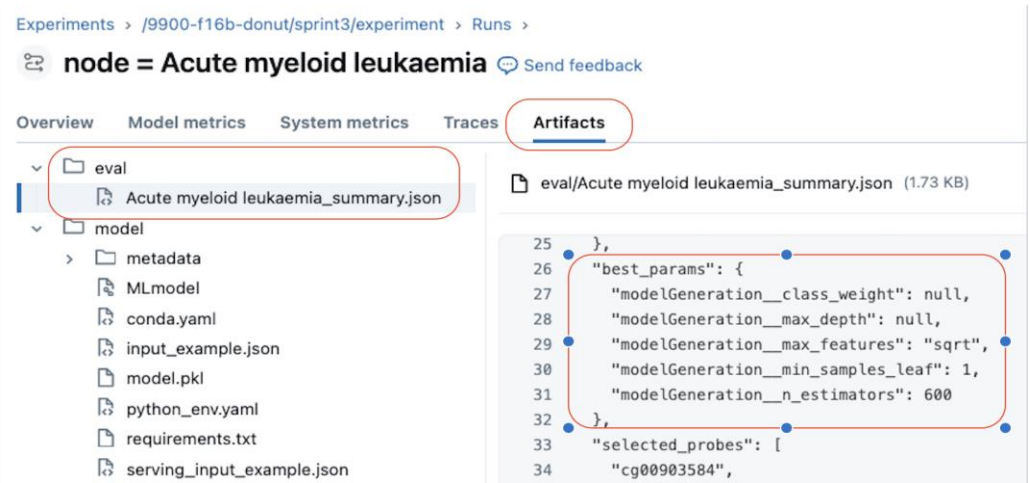

Figure 10 Trained model hyperparameter setting

Due to the need for specific environments and datasets for training results, these environments and

datasets are private to the client. So, the results may not be reproducible in other environments. The images in the steps already contain the expected results.

2 System Architecture Diagram

2.1 Key components on Databricks

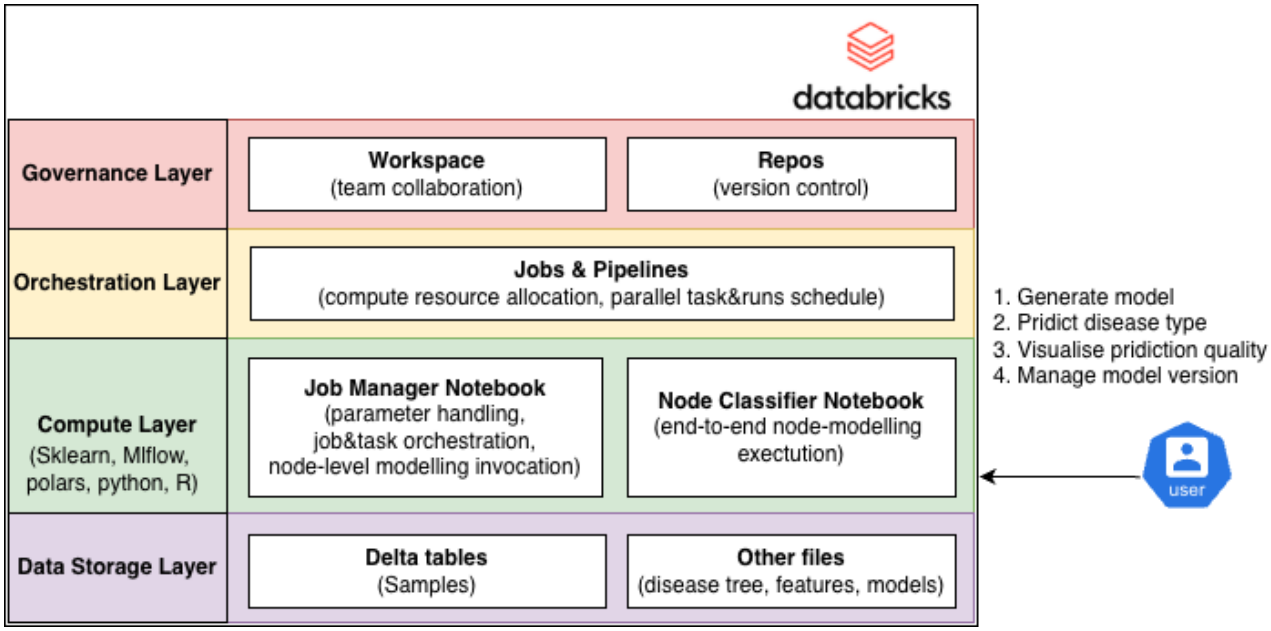

Figure 11 Project Design on Databricks

The overview of the architecture design is shown in Figure 11. The end-to-end disease classification system on the Databricks platform is structured into four layers:

- **Governance Layer:** provides Workspace and Repos for user-friendly and secure team collaboration and version control.
- **Orchestration Layer:** uses Jobs & Pipelines to handle parameter passing, manages compute resource allocation, schedules multiple node-level runs, and executes parallel tasks.
- **Compute Layer:** uses Scikit-learn, MLflow and Polars to implement core project functionality in Python, R and Notebook. Two main notebooks perform the core logic:
    1) Job Manager: handles parameters, triggers and launches Job and Task.
    2) Node Classifier: executes the end-to-end node-level modelling pipeline.
- **Data Storage Layer:** store all datasets and model-related files.

2.2 Workflows of the system

Data flows from the raw methylation dataset and the disease tree located in the data storage layer,

through feature selection and model training in the computation layer. It is then returned to the data storage layer and the MLflow model registry as the trained model and evaluation results.

2.2.1 Parallel modelling execution workflow

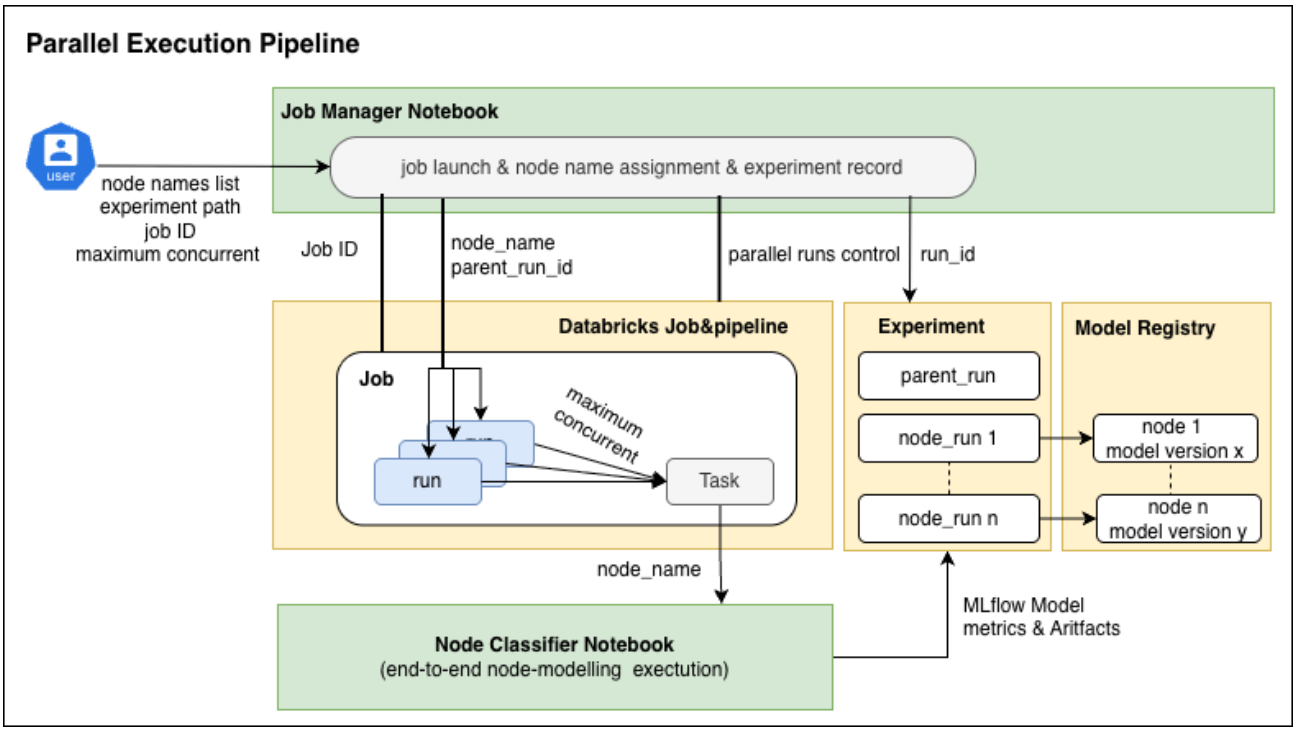

Figure 12 Parallel Execution Pipeline

The workflow of executing node-level parallel tasks in the Databricks environment is illustrated in Figure 12. The user first provides a list of node names, along with the Databricks Job ID, the target MLflow experiment path, and the maximum number of concurrent Job runs.

When the Job Manager Notebook is executed, it creates a parent MLflow run and iterates over the node list to submit one Databricks Job run per node. Each Job run invokes the same task, which subsequently launches the Node Classifier Notebook and passes in the node_name and parent_run_id arguments. The Databricks Job enforces its configured maximum concurrency setting to control how many node tasks may run in parallel, balancing execution speed and resource consumption.

For each node, the Node Classifier Notebook reads the node's training data from the Databricks File System (DBFS), performs the complete end-to-end modelling workflow, logs the trained model, evaluation metrics and related artifacts to the MLflow experiment and DBFS. The best-performing model at each node can then be registered as a new version in the MLflow Model Registry, providing a traceable version record for the node-level classifier. As a result, the experiment forms a structured hierarchy: a single parent run containing multiple node-level sub-runs, each corresponding to the model training process and results for a specific node, along with the associated registered model

version.

2.2.2 End-to-end single node modelling workflow

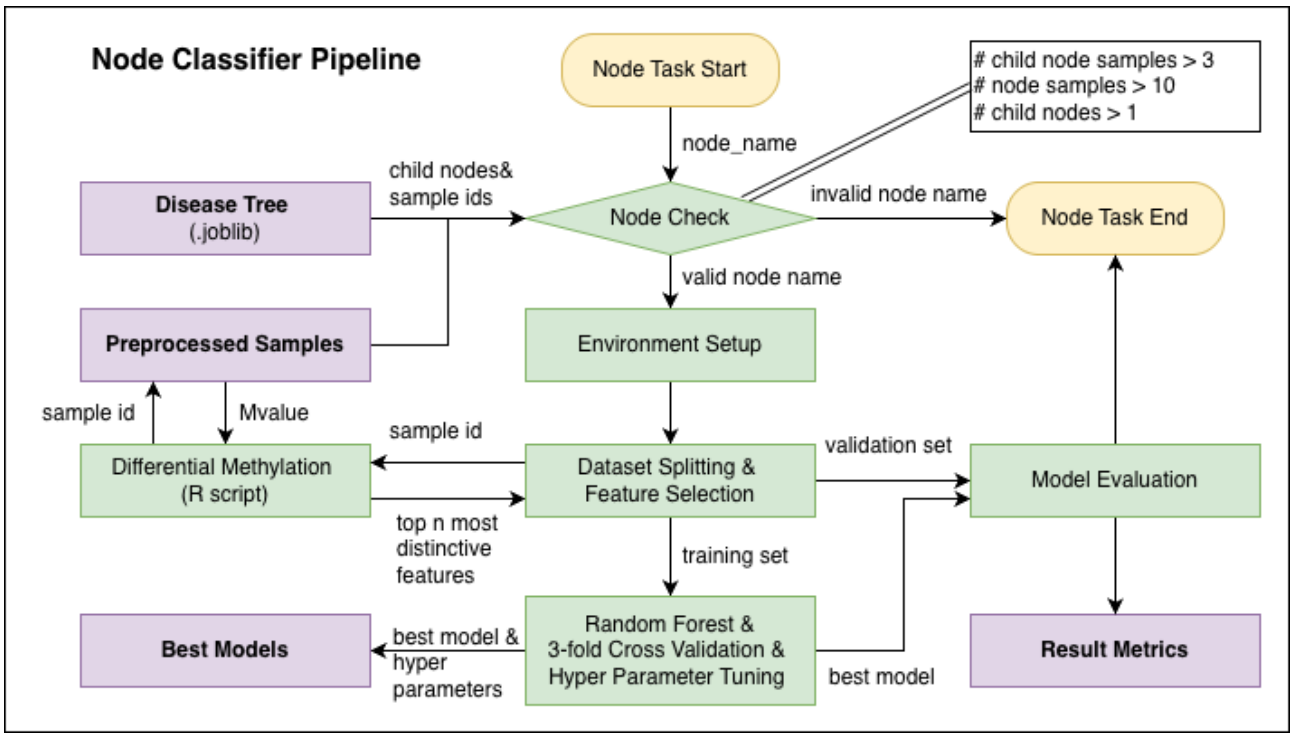

Figure 13 Node Classifier Pipeline

Figure 13 summarises the end-to-end pipeline for a single node task. A node task begins with a given node name and first performs a node check to validate the availability and quality of the associated data, ensuring that nodes with sparse training samples or nodes that do not require classification are filtered out. If the node does not meet the predefined criteria, the task terminates without proceeding further. For valid nodes, the pipeline continues with environment setup, followed by dataset splitting and feature selection, where the top N most distinctive features are selected to reduce computational cost and resource usage. The resulting training set is then passed to a Random Forest model with cross-validation and hyperparameter tuning to obtain the optimal model for that node. Finally, model evaluation is conducted on the validation set, and the resulting metrics are stored to support downstream analysis and cross-node comparison.

3 Design Justifications

1. Evolution of Requirements Across Sprints

   Throughout the project, our system design evolved substantially as our understanding of the requirements matured and as technical limitations became clearer. Initially, the requirement seemed simple:

- train a model for each disease type
- run each model manually
- track performance results

However, we then learn more about the client's requirement:

- the pipeline must support multiple disease types
- training must run repeatedly and reliably
- jobs should be automated, not manually triggered
- all runs must be tracked under a single task session
- execution must scale to additional nodes without redesign

These changing requirements directly influenced the evolution of our system architecture.

2. What Changed and Why
   a. Requirement Change: Automated pipeline orchestration
      Problem: The client expected a fully automated execution pipeline, not a UI-operated workflow.

      Change introduced: We implemented a Python orchestrator (job_manager.ipynb) to automatically launch Databricks job runs using the Jobs API.

      Why:
      - Removes manual triggers
      - Ensures reproducibility
      - Enables large-scale training across many nodes
      - Enables workflow scheduling if needed

   b. Requirement Change: Ability to group runs under one experiment session
      Problem: In the original design, separate runs had no shared lineage.
      This made comparison difficult and prevented aggregated analysis.

      Change introduced: We used MLflow to create a parent run, and each node run is executed as a child run using the parent run ID.

      Why:
      - All child runs appear under one session
      - Ensures clean experiment organisation
      - Supports cross-disease analytics

c. Requirement Change: Dynamic job parameters
Problem: Initially, job parameters (node name, experiment path) were hardcoded or manually passed.

Change introduced: We now pass parameters programmatically into the Databricks Job API

Why:
- Less human error
- Fully automated parameter flow
- Easy to plug in new parameters as the system grows

d. Requirement Change: Differential Methylation Script Redesign
Problem: The initial design did not support our new requirement for performance issue on Databricks

Change introduced: Modified differential methylation accepts in-memory matrices + labels via rpy2 and data frame as NumPy.

Why:
- Works seamlessly with our Databricks orchestrator and parallel node runs.
- Usage of NumPy instead of Pandas to get much faster execution.
- Enables multi-class modelling for all disease types.

e. Requirement Change: Centralized tracking of job submissions
Problem: Initially, the system used a training log to track the progress of training and print every time a model is trained.

Change introduced: We log submission metadata as an MLflow artifact.

Why:
- Gives full audit trail
- Supports debugging and validation
- Helps track run IDs programmatically

4 User-Driven Evaluation of Solution

4.1 Main Features of the System

This project delivers a complete hierarchical methylation-based classification system, designed to meet the functional requirements specified by the client. The system is structured around a disease ontology tree, where each internal node functions as an independent classification task. The following subsections describe how the solution satisfies each functional requirement and how it operates from a user perspective.

4.1.1 Node-Isolated Training Workflow

A native aspect of the solution is the capacity of treating every node of the paediatric cancer ontology as a node-isolated binary classification task. The DiseaseTree is combining the sample IDs of all children leaves, which allows deterministic generation of training and testing datasets for every node. This isolates the data leakage problem.

From a user's perspective, all that is required to trigger training is the loading of the frozen tree and execution of the splitting function. It results in the generation of node-wise training units, which then can be passed to the task generator.

4.1.2 Differential Methylation Feature Selection

To ensure biological relevance, the system integrates R-based differential methylation (DM) analysis. For each node, top probes are selected based on statistically significant methylation differences between child groups. This feature provides strong interpretability and enhances the classifier's ability to focus on meaningful signals in the data.

Feature selection happens automatically, as part of the training process, requiring no input or overhead from the user.

4.1.3 Hyperparameter Search and Model Training

Training of each node is performed by a Sklearn Pipeline implementation of Random Forest classifier. Hyperparameter search is managed by means of GridSearchCV, allowing exhaustive search of multiple parameter combinations: optimized tree depth, number of trees, tree sampling features, class weight, and other relevant options.

To guarantee the biological niche of feature selection, the implementation for the R based differential methylation (DM) is included in the training process. For every n level node, significant methylation differences between each pair of child groups are used for selection of top probes. It adds explainability and unconfirms noise factors regarding training variables.

Metric results, LDA probes used and all hyperparameters stored in MLflow tracking.

4.1.4 Parallel Execution Through Databricks Jobs

Implementation allows for parallel training and can be scaled to work with high numbers of nodes.

With the help of the Databricks Jobs API, a node can be processed as a separate job. Every model training is performed with a new job, meaning every model can be trained in parallel, reducing total execution time.

The node_modeling.ipynb notebook acts as a job entry point. It's responsible for environment setup, MLflow run start, imports and task execution.

4.1.5 Structured Evaluation and Model Registry

Under every completed job is an assessment for model performance. Results are stored in a machine-readable format: JSON file with all scores, and more accessible format: well-structured summary static page with accuracy, AUC, Brier score, confusion matrix, and size of validation set for model training, the number of samples, and selected probes.

Model is stored in MLflow's Model Registry, which allows for tracking of multiple model versions, re-producible application to downstream projects and integration with models and prediction pipelines.

4.1.6 Prediction for New Methylation Samples

The proposed solution includes prediction pipeline, which is aligning new methylation data to the probe list at every node level.

MLflow allows for easy loading and prediction with a stored model version. This will be relevant for setting up any downstream research tasks with new data.

4.2 Evaluation Criteria

From the perspective of clients/user-types (clinicians, biomedical researchers), the following criteria have been considered:

- **Classification accuracy and stability**
  Reliability of produced metrics.
- **Biological relevance**
  Whether there is a feature selection process that is backed by biological data.
- **Reproducibility and auditability**
  Whether training process can be re-created together with metadata, seeds and environment.
- **Scalability and computational efficiency**
  Can multiple node-level tasks run at the same time.
- **User practicality and clarity of outputs**

Whether summaries, metadata, models are user-friendly and accessible.

- **Maintainability and future extension**
  How easy it is to attach new diseases, models or datasets to the pipeline.

4.3 Evaluation of Effectiveness

Evaluation Criterion	Assessment Summary	Effectiveness Rating (1–5)	Notes / Justification
Accuracy and Reliability	Many nodes achieve strong accuracy, AUC and calibration. Nodes with limited samples show fewer stable results, due to data constraints rather than system design.	**4 (Strong)**	Metrics are consistently logged and well-structured; evaluation is robust but limited by sample size and absence of a dedicated test set.
Biological Validity	R-based differential methylation provides biologically meaningful feature selection tailored for each node.	**5 (Very Strong)**	Selected probes have interpretable biological grounding; satisfies client expectation for biologically informed modelling.
Reproducibility and Traceability	Frozen tree, deterministic splits, MLflow tracking, and stored evaluation summaries ensure complete reproducibility.	**5 (Very Strong)**	All metadata, seeds, probe lists, and environment settings are recorded, enabling repeatable runs.
Scalability and Performance	Parallel execution via Databricks Jobs enables simultaneous node training with controlled concurrency.	**4 (Strong)**	Although hyperparameter search uses *n_jobs=1*, overall workflow scales effectively across nodes.
Usability and Practical Value	Outputs (JSON/TXT reports + MLflow artefacts) are clear and structured for researchers.	**4 (Strong)**	System is accessible for technical users; clinician-facing interface is outside project scope.
Maintainability and Extensibility	Modular code structure allows for easy replacement or extension of components (e.g., classifiers, DM methods).	**5 (Very Strong)**	Architecture supports future expansion with minimal refactoring.

4.4 Unmet or Partially Met Objectives

The solution achieves most of stated goals, apart from a few, which are partially met:

- No dedicated test set
  There are only train/validation sets established. Hold-out test is an established practice to measure model's generalization ability.
- No clinical user interface
  Outputs are aimed at researchers. Clinicians need a different UX.
- Hyperparameter search is not distributed
  The search is performed with the same node on n_jobs=1 to ensure stability of results. It's inefficient for very large datasets.

The limitations don't undermine utility, albeit providing opportunity for improvements. These facts need to be elaborated in the report's Limitations section.

5 Limitations and Future Work

5.1 Limitations

While we successfully established a scalable pipeline for pediatric cancer methylation classification with Databricks, our project has certain limitations impacting its performance and usage. One key constraint is the limited compute power provided by Databricks, which restricts the scale of parallel computations during very large dataset training or complex disease tree deployment, as indicated in the proposal's discussion on distributed systems (including Spark) that prioritize scaling but require significant customization. Despite achieving ~40% improvements on internal benchmarks, we could still encounter scalability issues when training on extensive datasets since our proposed premium cluster subscription was only up for a 10-day trial run. Furthermore, since we encapsulated model training within the pipeline, overall training times are significantly increased. Each node's operations include several sequential steps such as train/test splitting and hyperparameter tuning, thus accumulating delays in a hierarchical pipeline setup, like the sequential data operations explored in the proposal's background section. Therefore, this design poses limitations on the reactivity of real-time updates or frequent retraining, which would not be ideal for clinical usage that requires rapid iterations.

5.2 Future Work

To overcome the limitations and based on client feedback from our demo, we plan to implement several upgrades to enhance the pipeline with robustness and efficiency. First, we will add more metrics logs to showcase accuracy, confusion matrices, and training progress. This suggested future improvement

will not only provide a more intuitive display of the training process and results but also allow clinicians and teams to easily monitor and audit their working processes, which satisfies the proposal's PPT's suggestions on improving documentation practices and providing reproducible artifacts. Moreover, we intend to optimize the existing R code for operations on loading the data. We can possibly achieve ~50% speed-up through modifications by pre-splitting datasets and comparing tree structures before adding new data, which reflects the proposal's PPT's conclusion on splitting datasets beforehand to improve efficiency. Future iterations may also continue to implement innovative improvements based on the proposal's exploration of imbalanced data training and reproducibility. Additional techniques such as automated class-aware balancing or cloud-agnostic deployment options to support transitioning to larger datasets and more complex hierarchies can be further integrated, thereby ensuring the reliability of the system as a decision-support tool for timely pediatric cancer diagnoses without compromising the high accuracy we demonstrated at nodes such as Leukaemia (93.8%) and Sarcoma (93.4%).